Reinforcement Learning: Session 1

From Markov Decision Processes to your first agent

Pegah Alizadeh
pegah.alizadeh@univ-lyon2.fr

Reinforcement Learning Course,
University of Lumière Lyon2, 2026

How this atelier works

Format

- 4 sessions: 2h (online) + 3× 4h (in person)
- Short theory blocks, then **you code**
- Work in pairs (or alone if you prefer)
- No exam: you learn by building agents

Each hands-on block

1. I pilot the first part live
2. We solve the first TODO together
3. You finish the notebook
4. Debrief: 2–3 pairs present their results

Notebook levels

- **Core**: everyone finishes this
- **Stretch**: if you have time
- **Bonus**: for the curious

Roadmap

1. MDPs & planning (today)
2. Tabular RL: Q-learning, SARSA
3. Deep RL: DQN from scratch
4. Policy gradient, PPO, RL for LLMs

Sessions 3–4: **LunarLander leaderboard**

Today

Time	Activity
0–10 min	Welcome, a teaser agent starts training
10–50 min	Theory: the RL problem, MDPs, Bellman equations
50–65 min	Live demo: the Gymnasium API
65–100 min	Breakout rooms: value iteration on FrozenLake
100–115 min	Debrief
115–120 min	Setup for next session

Goal of today: understand *why* the Bellman equations hold, and turn one of them into an algorithm that computes an optimal policy.

Textbooks

- An Introduction to Reinforcement Learning, Richard S. Sutton and Andrew G. Barto, 2018 (Sutton & Barto)
  - MIT press book, freely available online: `http://incompleteideas.net/book/the-book-2nd.html`
  - code examples of the book
- Algorithms for Reinforcement Learning, Csaba Szepesvári, 2010 (Szepesvári)
  - Morgan and Claypool book, freely available online:
  - `https://sites.ualberta.ca/~szepesva/papers/RLAlgsInMDPs.pdf`
- Reinforcement Learning: An Overview, Kevin Murphy, 2024 (regularly updated, covers modern deep RL and RL for LLMs)
  - `https://arxiv.org/abs/2412.05265`

Resources: courses and videos

- **David Silver's lectures** (these slides are based on them), video recordings: `https://www.youtube.com/playlist?list=PLqYmG7hTraZDM-OYHWgPebj2MfCFzFObQ`
- **Hugging Face Deep RL course**: free, hands-on, notebooks: `https://huggingface.co/learn/deep-rl-course`
- **OpenAI Spinning Up**: a clear introduction to deep RL algorithms: `https://spinningup.openai.com`
- **Berkeley CS285** (Sergey Levine), deep RL, more advanced: `https://rail.eecs.berkeley.edu/deeprlcourse/`
- **Stanford CS234** (Emma Brunskill): `https://web.stanford.edu/class/cs234/`
- **RLHF book** (Nathan Lambert), RL for language models: `https://rlhfbook.com`

Resources: code and libraries

- **Gymnasium**: the standard API for RL environments (we use it today):
  `https://gymnasium.farama.org`
- **CleanRL**: single-file, readable implementations of DQN, PPO, ...
  `https://github.com/vwxyzjn/cleanrl`
- **Stable-Baselines3**: reliable implementations, train an agent in 3 lines:
  `https://stable-baselines3.readthedocs.io`
- **RL Baselines3 Zoo**: tuned hyperparameters and pre-trained agents:
  `https://github.com/DLR-RM/rl-baselines3-zoo`
- **The 37 implementation details of PPO**: why "the same algorithm" gives different results:
  `https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/`
- **Andrej Karpathy, "Pong from Pixels"**: policy gradient explained in a blog post:
  `https://karpathy.github.io/2016/05/31/rl/`

Characteristics of Reinforcement Learning

What makes reinforcement learning different from other machine learning paradigms?

- There is no supervisor, only a *reward* signal
- Feedback is delayed, not instantaneous
- Time really matters (sequential, non i.i.d data)
- Agent's actions affect the subsequent data it receives

Where you meet RL today

Games and search

- AlphaGo / AlphaZero: Go, chess, shogi (2016–2017)
- MuZero: plans with a *learned* model (2020)

Control and robotics

- Legged robots trained in simulation, transferred to the real world
- Plasma control in a nuclear fusion reactor (DeepMind, 2022)

Language models

- RLHF: ChatGPT is fine-tuned with RL from human preferences (2022)
- Reasoning models (e.g. DeepSeek-R1, 2025) are trained with RL on verifiable rewards, using GRPO, a policy gradient method

Algorithms

- AlphaDev: faster sorting routines found by RL (2023)

All of them rest on the same foundation: **the Markov Decision Process**.

Rewards

- A reward R_t is a scalar feedback signal
- Indicates how well agent is doing at step t
- The agent's job is to maximise cumulative reward

Reinforcement learning is based on the reward hypothesis

Definition (Reward Hypothesis)

All goals can be described by the maximisation of expected cumulative reward

Do you agree with this statement?

Major Components of an RL Agent

- An RL agent may include one or more of these components:
  - Policy: agent's behaviour function
  - Value function: how good is each state and/or action
  - Model: agent's representation of the environment

Policy

- A policy is the agent's behaviour
- It is a map from state to action, e.g.
- Deterministic policy: $a = \pi(s)$
- Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$

Value Function

- Value function is a prediction of future reward
- Used to evaluate the goodness/badness of states
- And therefore to select between actions, e.g.

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... \mid S_t = s \right]$$

Example: Value Function in Practice

Same concept, different state representations

- In a maze: a state s is the agent's position
- In Atari: a state s is a high-dimensional observation (pixels)



In both cases, the value function answers the same question:

$V^{\pi}(s)$ = expected future reward starting from s

- Large problems: value learned with function approximation
- Small problems: value stored explicitly (tables)

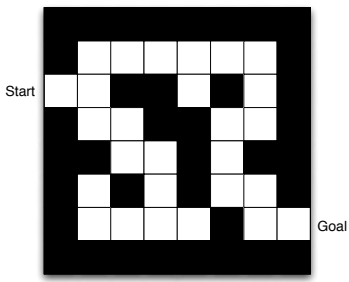
Next: a simple maze where we can see values explicitly.

Model

- A model predicts what the environment will do next
- $\mathcal{P}$ predicts the next state
- $\mathcal{R}$ predicts the next (immediate) reward, e.g.

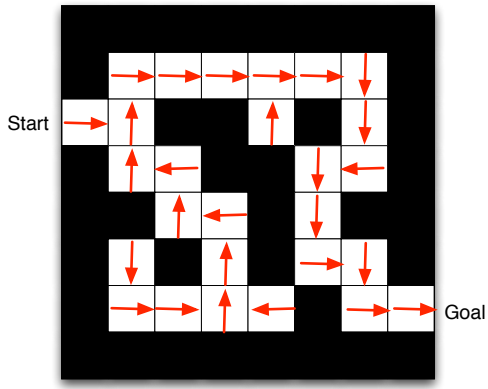
$$\mathcal{P}^a_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$$
$$\mathcal{R}^a_s = \mathbb{E}\left[R_{t+1} \mid S_t = s, A_t = a\right]$$

Maze Example



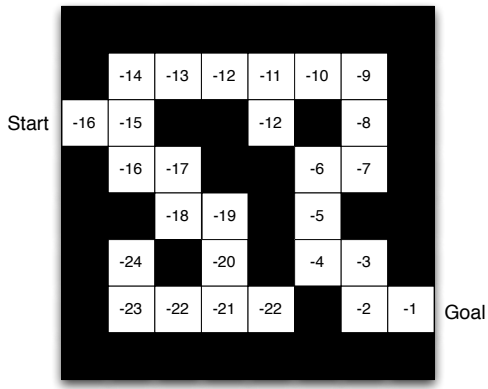
- Rewards: -1 per time-step
- Actions: N, E, S, W
- States: Agent's location

Maze Example: Policy



- Arrows represent policy $\pi(s)$ for each state s

Maze Example: Value Function



- Numbers represent value $v_\pi(s)$ of each state s

Learning and Planning

Two fundamental problems in sequential decision making

- Reinforcement Learning:
  - The environment is initially unknown
  - The agent interacts with the environment
  - The agent improves its policy
- Planning:
  - A model of the environment is known
  - The agent performs computations with its model (without any external interaction)
  - The agent improves its policy
  - a.k.a. deliberation, reasoning, introspection, pondering, thought, search

Concept check

1. In chess, what would you choose as the reward? Why is "+1 for capturing a piece" a dangerous choice?
2. A robot vacuum only sees what is in front of it. Is its observation a *state* (Markov)? What could you add to make it closer to one?
3. You know the rules of a board game perfectly. Is finding a good strategy a *learning* or a *planning* problem?
4. Value function or policy: which one do you need to *act*? Which one tells you *how good* a situation is?

Markov Property

"The future is independent of the past given the present"

Definition

A state S_t is *Markov* if and only if

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, ..., S_t]$$

- The state captures all relevant information from the history
- Once the state is known, the history may be thrown away
- i.e. The state is a sufficient statistic of the future

Return

Definition

The *return* G_t is the total discounted reward from time-step t.

$$G_t = R_{t+1} + \gamma R_{t+2} + ... = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The *discount* $\gamma \in [0, 1]$ is the present value of future rewards
- The value of receiving reward R after $k + 1$ time-steps is $\gamma^k R$.
- This values immediate reward above delayed reward.
  - γ close to 0 leads to "myopic" evaluation
  - γ close to 1 leads to "far-sighted" evaluation

Markov Decision Process

A Markov decision process (MDP) is a Markov reward process with decisions. It is an *environment* in which all states are Markov.

Definition

A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}^{a}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s, A_t = a\right]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}^{a}_{s} = \mathbb{E}\left[R_{t+1} \mid S_t = s, A_t = a\right]$
- γ is a discount factor $\gamma \in [0, 1]$.

Policies (1)

Definition

A *policy* π is a distribution over actions given states,

$$\pi(a|s) = \mathbb{P}\left[A_t = a \mid S_t = s\right]$$

- A policy fully defines the behaviour of an agent
- MDP policies depend on the current state (not the history)
- i.e. Policies are *stationary* (time-independent),
  $A_t \sim \pi(\cdot|S_t), \forall t > 0$

Value Function

Definition

The *state-value function* $v_\pi(s)$ of an MDP is the expected return starting from state s, and then following policy π

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right]$$

Definition

The *action-value function* $q_\pi(s, a)$ is the expected return starting from state s, taking action a, and then following policy π

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right]$$

Bellman Expectation Equation

The state-value function can again be decomposed into immediate reward plus discounted value of successor state,

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s \right]$$

The action-value function can similarly be decomposed,

$$q_\pi(s, a) = \mathbb{E}_\pi \left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a \right]$$

Where does the Bellman equation come from?

The return satisfies a simple recursion:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = R_{t+1} + \gamma\big(R_{t+2} + \gamma R_{t+3} + \dots\big) = R_{t+1} + \gamma\, G_{t+1}$$

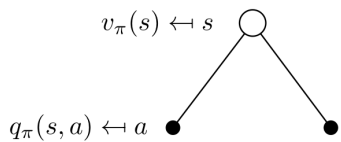
Take the expectation under π, given $S_t = s$:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma\, G_{t+1} \mid S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma\, v_\pi(S_{t+1}) \mid S_t = s]$$

The last step uses the **Markov property**: once we know S_{t+1}, the past does not matter, so $\mathbb{E}[G_{t+1} \mid S_{t+1}] = v_\pi(S_{t+1})$.

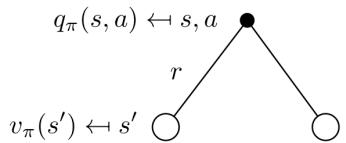
Interpretation: value of a state = immediate reward + discounted value of where you land.
The next slides write this expectation out explicitly with π, $\mathcal{P}$ and $\mathcal{R}$.

Bellman Expectation Equation for V^π



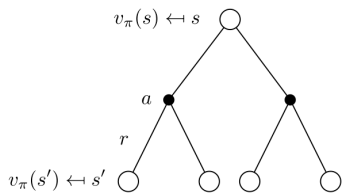
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s,a)$$

Bellman Expectation Equation for Q^π



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

Bellman Expectation Equation for v_π (2)



$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right)$$

Optimal Value Function

Definition

The *optimal state-value function* $v_*(s)$ is the maximum value function over all policies

$$v_*(s) = \max_\pi v_\pi(s)$$

The *optimal action-value function* $q_*(s, a)$ is the maximum action-value function over all policies

$$q_*(s, a) = \max_\pi q_\pi(s, a)$$

- The optimal value function specifies the best possible performance in the MDP.
- An MDP is "solved" when we know the optimal value fn.

Optimal Policy

Define a partial ordering over policies

$$\pi \geq \pi' \text{ if } v_\pi(s) \geq v_{\pi'}(s), \forall s$$

Theorem

For any Markov Decision Process

- *There exists an optimal policy π_* that is better than or equal to all other policies, $\pi_* \geq \pi, \forall \pi$*
- *All optimal policies achieve the optimal value function, $v_{\pi_*}(s) = v_*(s)$*
- *All optimal policies achieve the optimal action-value function, $q_{\pi_*}(s, a) = q_*(s, a)$*

Finding an Optimal Policy

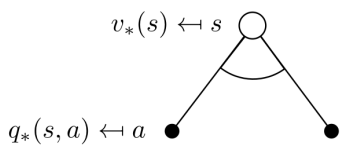
An optimal policy can be found by maximising over $q_*(s, a)$,

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\text{argmax}}\ q_*(s, a) \\ 0 & \textit{otherwise} \end{cases}$$

- There is always a deterministic optimal policy for any MDP
- If we know $q_*(s, a)$, we immediately have the optimal policy

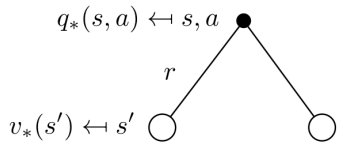
Bellman Optimality Equation for v_*

The optimal value functions are recursively related by the Bellman optimality equations:



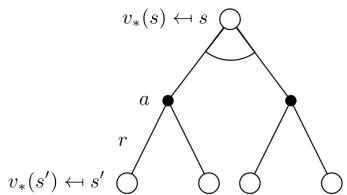
$$v_*(s) = \max_a q_*(s,a)$$

Bellman Optimality Equation for Q^*



$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

Bellman Optimality Equation for V^* (2)



$$v_*(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

Solving the Bellman Optimality Equation

- Bellman Optimality Equation is non-linear
- No closed form solution (in general)
- Many iterative solution methods
  - Value Iteration
  - Policy Iteration
  - Q-learning
  - Sarsa

Deterministic Value Iteration

- If we know the solution to subproblems $v_*(s')$
- Then solution $v_*(s)$ can be found by one-step lookahead

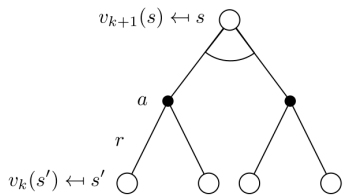
$$v_*(s) \leftarrow \max_{a \in \mathcal{A}} \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

- The idea of value iteration is to apply these updates iteratively
- Intuition: start with final rewards and work backwards
- Still works with loopy, stochastic MDPs

Value Iteration

- Problem: find optimal policy π
- Solution: iterative application of Bellman optimality backup
- $v_1 \rightarrow v_2 \rightarrow ... \rightarrow v_*$
- Using synchronous backups
  - At each iteration $k + 1$
  - For all states $s \in \mathcal{S}$
  - Update $v_{k+1}(s)$ from $v_k(s')$
- Convergence to v_* will be proven later
- Unlike policy iteration, there is no explicit policy
- Intermediate value functions may not correspond to any policy

Value Iteration (2)



$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

$$\mathbf{v}_{k+1} = \max_{a \in \mathcal{A}} \boldsymbol{\mathcal{R}}^{\boldsymbol{a}} + \gamma \boldsymbol{\mathcal{P}}^{\boldsymbol{a}} \mathbf{v}_k$$

Why does value iteration converge?

Write one sweep of value iteration as an operator T acting on a value vector V:

$$(TV)(s) = \max_a \left(\mathcal{R}_s^a + \gamma \sum_{s'} \mathcal{P}_{ss'}^a V(s') \right)$$

Key fact (γ-contraction): for any two value vectors U, V,

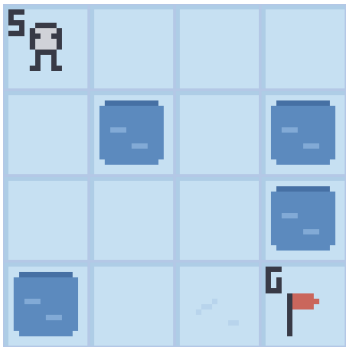
$$\| TU - TV \|_\infty \;\leq\; \gamma \, \| U - V \|_\infty$$

Consequences

- v_* is the unique fixed point: $Tv_* = v_*$ (the Bellman optimality equation)
- The error shrinks by a factor γ at each sweep: $\| V_k - v_* \|_\infty \leq \gamma^k \| V_0 - v_* \|_\infty$
- Reaching precision ε takes at most about $\log(1/\varepsilon) / \log(1/\gamma)$ sweeps: the bound for $\gamma = 0.99$ is $\approx 10\times$ larger than for $\gamma = 0.9$

Proof: Silver Lecture 3, "Contraction Mapping" section (slides 35–42).

From the MDP to code: Gymnasium



FrozenLake 4×4: reach the goal, avoid the holes.

Full description: Gymnasium docs, FrozenLake

MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ in Gymnasium

MDP	Gymnasium
$\lvert\mathcal{S}\rvert$	`env.observation_space.n`
$\lvert\mathcal{A}\rvert$	`env.action_space.n`
$\mathcal{P}, \mathcal{R}$	`env.unwrapped.P[s][a]`
γ	chosen by *you*

`P[s][a]` is a list of tuples:

```
[(prob, next_state, reward, terminated), ...]
```

Learning agents (next sessions) only use `env.reset()` and `env.step(a)`: they never see `P`.

Image: rendered with Gymnasium (Farama Foundation, MIT license).

Value iteration: from equation to code

Bellman optimality backup (one-step lookahead):

$$V_{k+1}(s) = \max_a \sum_{s'} \mathcal{P}^a_{ss'} \big(r + \gamma\, V_k(s')\big)$$

```
V = zeros(n_states)
repeat:
    for each state s:
        Q[s, a] = sum over (p, s2, r, done) in P[s][a] of
                  p * (r + gamma * V[s2] * (not done))
    V_new = max over a of Q[s, a]
    stop when max |V_new - V| < theta
    V = V_new
policy[s] = argmax over a of Q[s, a]      # greedy policy
```

Why `(not done)`? A terminal state has no future: its value is 0.

Your turn: value iteration on FrozenLake

Notebook: `session-1/s1_value_iteration.ipynb`: `git pull`, then `jupyter lab`

`https://github.com/pegahani/Atelier-thematique-Principles-of-Reinforcement-Learning`

Breakout rooms: pairs, 35 minutes. Post your results in the chat.

- **Core**
  - Explore the environment, measure a random agent
  - Implement value iteration, extract the greedy policy
  - Check: $V(\text{start}) = 0.59049$ for $\gamma = 0.9$, non-slippery. Why exactly 0.9^5?
- **Stretch**
  - Implement policy iteration and compare the number of iterations
- **Bonus**
  - Slippery ice: why does the optimal policy sometimes walk *away* from the goal?
  - Bigger map (8×8), effect of γ

Debrief

1. On slippery ice with $\gamma = 0.9$, $V(\text{start}) \approx 0.07$, yet the agent reaches the goal $\approx 73\%$ of the time. Is there a contradiction?
2. Value iteration needed ≈ 110 sweeps for $\gamma = 0.9$ and ≈ 440 for $\gamma = 0.99$ (slippery). The contraction bound predicts a ratio of ≈ 10, we observe ≈ 4. Why is the bound pessimistic?
3. Value iteration needs P. In most real problems we do not have it. What could we do instead? $\rightarrow$ **Session 2**

Before Session 2

1. **Finish your setup**: every cell of `setup/check_setup.ipynb` must pass (it also checks PyTorch and LunarLander, which we need from Session 2). Still failing? Bring the error message to Session 2, we fix it together.
2. **Solution notebook**: `git pull` after today's session.
3. **Optional reading**: Sutton & Barto, Chapters 3 and 4.
4. **Optional video**: David Silver, Lectures 2 and 3.

Next session: learning *without* a model: Monte Carlo, TD, SARSA and Q-learning.

Attribution

Reordered slides from David Silver's UCL RL course materials (CC-BY-NC 4.0).

Appendix: additional slides

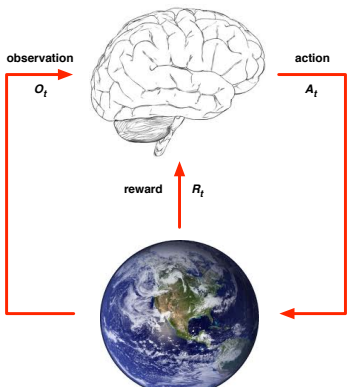
Examples of Reinforcement Learning

- Fly stunt manoeuvres in a helicopter
- Defeat the world champion at Backgammon
- Manage an investment portfolio
- Control a power station
- Make a humanoid robot walk
- Play many different Atari games better than humans

Sequential Decision Making

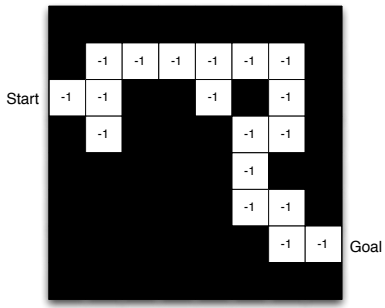
- Goal: *select actions to maximise total future reward*
- Actions may have long term consequences
- Reward may be delayed
- It may be better to sacrifice immediate reward to gain more long-term reward
- Examples:
  - A financial investment (may take months to mature)
  - Refuelling a helicopter (might prevent a crash in several hours)
  - Blocking opponent moves (might help winning chances many moves from now)

Agent and Environment



- At each step t the agent:
  - Executes action A_t
  - Receives observation O_t
  - Receives scalar reward R_t
- The environment:
  - Receives action A_t
  - Emits observation O_{t+1}
  - Emits scalar reward R_{t+1}
- t increments at env. step

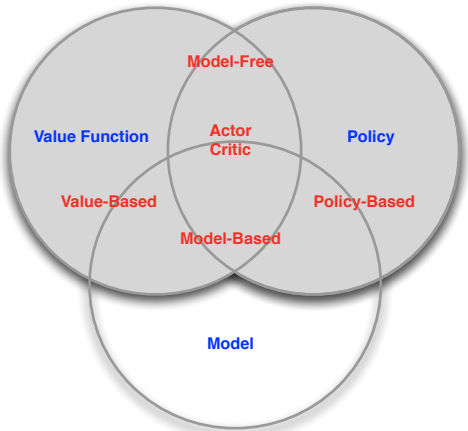
Maze Example: Model



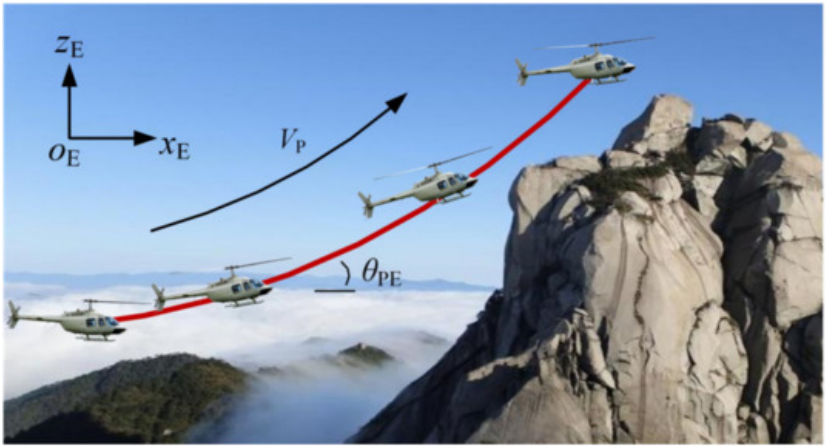
- Agent may have an internal model of the environment
- Dynamics: how actions change the state
- Rewards: how much reward from each state
- The model may be imperfect

- Grid layout represents transition model $\mathcal{P}^a_{ss'}$
- Numbers represent immediate reward $\mathcal{R}^a_s$ from each state s (same for all a)

RL Agent Taxonomy

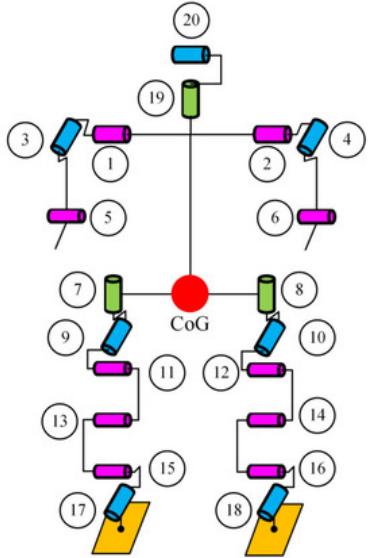


Helicopter Manoeuvres



Humanoid robot





Introduction to MDPs

- *Markov decision processes* formally describe an environment for reinforcement learning
- Where the environment is *fully observable*
- i.e. The current *state* completely characterises the process
- Almost all RL problems can be formalised as MDPs, e.g.
  - Optimal control primarily deals with continuous MDPs
  - Partially observable problems can be converted into MDPs
  - Bandits are MDPs with one state

Markov Reward Process

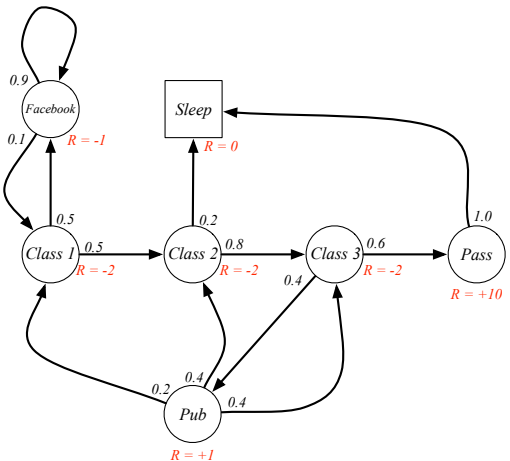
A Markov reward process is a Markov chain with values.

Definition

A *Markov Reward Process* is a tuple $\langle \mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s\right]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s = \mathbb{E}\left[R_{t+1} \mid S_t = s\right]$
- γ is a discount factor, $\gamma \in [0, 1]$

Example: Student MRP

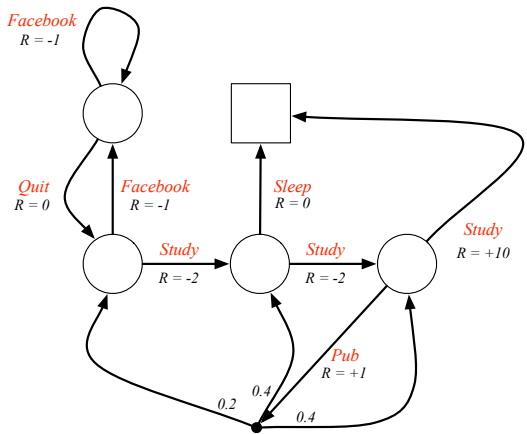


Why discount?

Most Markov reward and decision processes are discounted. Why?

- Mathematically convenient to discount rewards
- Avoids infinite returns in cyclic Markov processes
- Uncertainty about the future may not be fully represented
- If the reward is financial, immediate rewards may earn more interest than delayed rewards
- Animal/human behaviour shows preference for immediate reward
- It is sometimes possible to use *undiscounted* Markov reward processes (i.e. $\gamma = 1$), e.g. if all sequences terminate.

Example: Student MDP



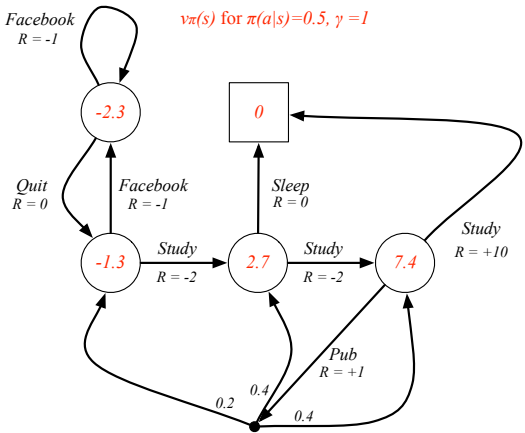
Policies (2)

- Given an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ and a policy π
- The state sequence $S_1, S_2, ...$ is a Markov process $\langle \mathcal{S}, \mathcal{P}^\pi \rangle$
- The state and reward sequence $S_1, R_2, S_2, ...$ is a Markov reward process $\langle \mathcal{S}, \mathcal{P}^\pi, \mathcal{R}^\pi, \gamma \rangle$
- where

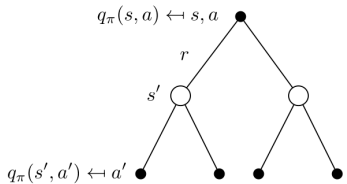
$$\mathcal{P}^\pi_{s,s'} = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{P}^a_{ss'}$$

$$\mathcal{R}^\pi_s = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{R}^a_s$$

Example: State-Value Function for Student MDP

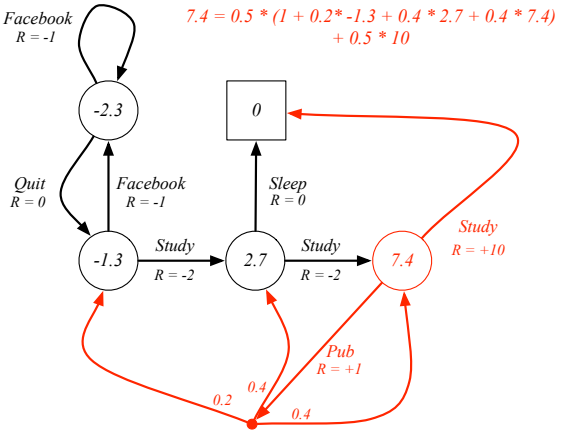


Bellman Expectation Equation for q_π (2)



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s',a')$$

Example: Bellman Expectation Equation in Student MDP



Bellman Expectation Equation (Matrix Form)

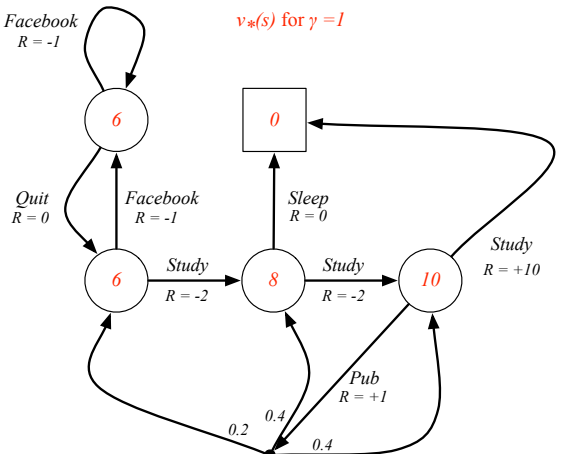
The Bellman expectation equation can be expressed concisely using the induced MRP,

$$v_\pi = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi v_\pi$$

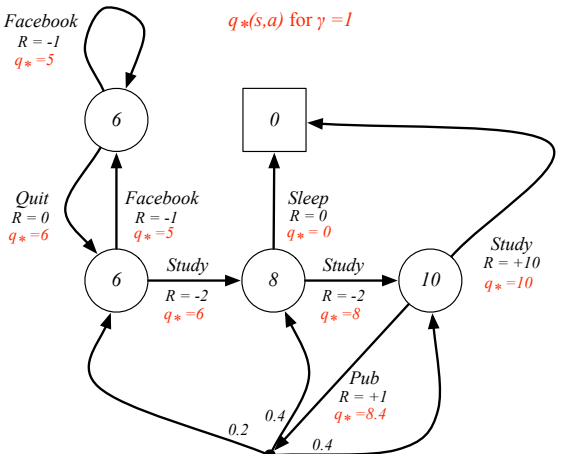
with direct solution

$$v_\pi = (I - \gamma \mathcal{P}^\pi)^{-1} \mathcal{R}^\pi$$

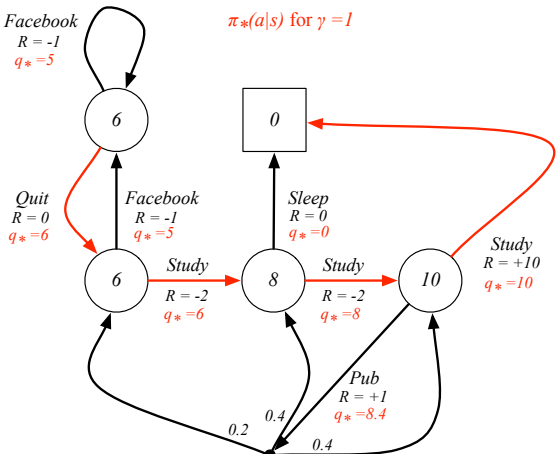
Example: Optimal Value Function for Student MDP



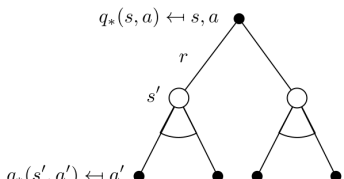
Example: Optimal Action-Value Function for Student MDP



Example: Optimal Policy for Student MDP



Bellman Optimality Equation for Q^* (2)



$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s',a')$$

Example: Bellman Optimality Equation in Student MDP

